Modelos Generativos Profundos

Autoencoders variacionales, flujos normalizantes, GANs y modelos basados en energía

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule

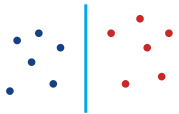
Discriminativo vs. Generativo

Modelo Discriminativo

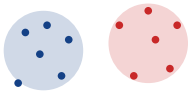
- Aprende $p(y \mid x)$: una frontera de decisión.
- Clasificación, detección, segmentación (clases 2.1–2.6).
- No sabe "cómo se ve" un dato.

Modelo Generativo

- Aprende $p(x)$ (o $p(x \mid y)$): la distribución de los datos.
- Permite **muestrear** datos nuevos $x \sim p_\theta$.
- Permite **evaluar** qué tan probable es un x.



$p(y \mid x)$: frontera



$p(x \mid y)$: densidad

Idea Central

Un modelo generativo $p_\theta(x)$ debe asignar **alta probabilidad** a datos reales y **baja** a todo lo demás, en espacios de altísima dimensión (una imagen $256 \times 256 \times 3$ vive en $\mathbb{R}^{196\,608}$).

¿Para Qué Sirve un Modelo Generativo?

Muestrear

- Síntesis de imágenes, audio, texto.
- Datos sintéticos para entrenar (clase 2.5).
- Aumentación de datos médicos escasos.

Evaluar $p_\theta(x)$

- Detección de anomalías: $p_\theta(x)$ bajo $\Rightarrow$ atípico.
- Compresión (Shannon: $-\log_2 p(x)$ bits).
- Comparar modelos (log-verosimilitud).

Representar

- Variables latentes z como **representación** compacta.
- Interpolación y edición semántica.
- Semi-supervisado, aprendizaje de features.

Hoy

Cuatro familias que resuelven el mismo problema con distintos compromisos: **VAE** (verosimilitud aproximada), **flujos normalizantes** (verosimilitud exacta), **GAN** (implícito, adversario) y **modelos basados en energía** (densidad no normalizada).

Máxima Verosimilitud = Minimizar KL

Dado $\mathcal{D} = \{x^{(i)}\}_{i=1}^{N} \sim p_{\text{data}}$, buscamos θ tal que $p_\theta \approx p_{\text{data}}$.

$$\theta^* = \arg\max_\theta \frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x^{(i)}) \xrightarrow{N\to\infty} \arg\max_\theta \mathbb{E}_{p_{\text{data}}}\big[\log p_\theta(x)\big] = \arg\min_\theta \mathrm{KL}\big(p_{\text{data}} \,\|\, p_\theta\big)$$

Por qué

$$\mathrm{KL}(p_{\text{data}} \| p_\theta) = \underbrace{\mathbb{E}_{p_{\text{data}}}[\log p_{\text{data}}]}_{\text{no depende de } \theta} - \mathbb{E}_{p_{\text{data}}}[\log p_\theta]$$

Dos requisitos que chocan

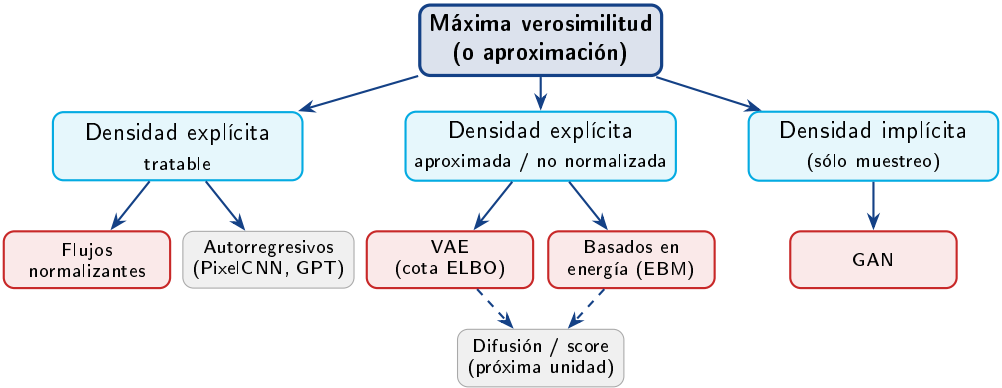
1. p_θ **flexible**: redes profundas.
2. p_θ **normalizada**: $\int p_\theta(x)\, dx = 1$.

Una red arbitraria $f_\theta(x) \geq 0$ no integra 1.

El Problema Central

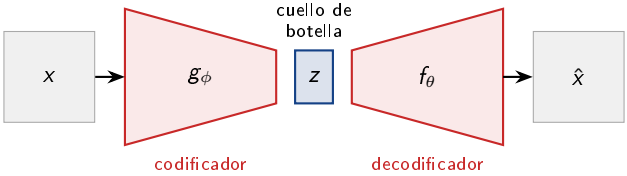
Cada familia de modelos generativos es, en el fondo, una **estrategia distinta para lidiar con la normalización**.

Taxonomía de Modelos Generativos Profundos



En rojo: lo que veremos hoy. La difusión combina ideas del VAE (variable latente jerárquica) y de los EBM (score $\nabla_x \log p$).

Autoencoder: Comprimir y Reconstruir



$$\phi^*, \theta^* = \arg\min_{\phi,\theta} \frac{1}{N} \sum_{i=1}^{N} \left\| x^{(i)} - f_\theta\left(g_\phi(x^{(i)})\right) \right\|^2, \qquad z = g_\phi(x) \in \mathbb{R}^d,\ d \ll D$$

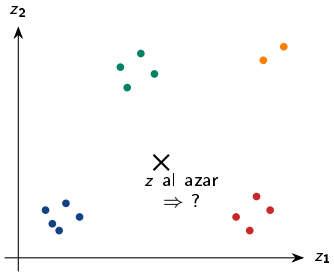
Qué aprende

- Con f, g lineales: equivale a **PCA**.
- No lineal: una variedad (*manifold*) de baja dimensión.
- Usos: compresión, denoising, anomalías.

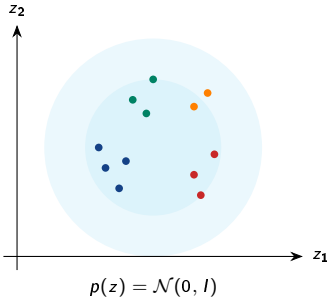
¿Es un modelo generativo?

No todavía. Si muestreamos z al azar y decodificamos, $f_\theta(z)$ suele ser basura: el espacio latente tiene "huecos" y no sabemos **de qué distribución** sacar z.

El Espacio Latente de un Autoencoder



Autoencoder determinista: códigos dispersos, regiones vacías, escala arbitraria.

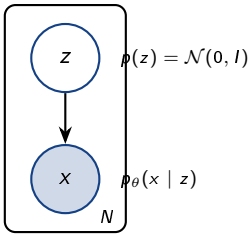


VAE: los códigos se fuerzan a cubrir una distribución conocida $\Rightarrow$ podemos muestrear $z \sim p(z)$.

Lo que falta

Un **modelo probabilístico** explícito: una distribución a priori $p(z)$ fácil de muestrear y una verosimilitud $p_\theta(x \mid z)$. Eso es el **autoencoder variacional**.

Modelo de Variable Latente



Proceso generativo

1. $z \sim p(z) = \mathcal{N}(0, I_d)$
2. $x \sim p_\theta(x \mid z) = \mathcal{N}\big(x;\, f_\theta(z),\, \sigma^2 I\big)$
   (o Bernoulli para imágenes binarias)

f_θ es una red neuronal: el **decodificador**.

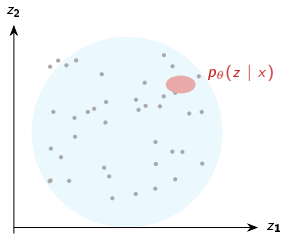
$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz \qquad \text{(mezcla infinita de gaussianas)}$$

Problema

La integral es **intratable**: f_θ es no lineal y z tiene decenas o cientos de dimensiones. Tampoco podemos calcular el posterior $p_\theta(z \mid x) = p_\theta(x \mid z)p(z)/p_\theta(x)$, que necesitaríamos para EM.

¿Por Qué no Monte Carlo Ingenuo?

$$p_\theta(x) = \mathbb{E}_{z\sim p(z)}\big[p_\theta(x \mid z)\big] \approx \frac{1}{S}\sum_{s=1}^{S} p_\theta(x \mid z^{(s)}), \qquad z^{(s)} \sim \mathcal{N}(0, I)$$



El problema

- Para un x dado, sólo una región **diminuta** de z explica x (en rojo).
- Casi todas las muestras del prior dan $p_\theta(x \mid z) \approx 0$.
- Varianza enorme; S crece exponencialmente con d.

Idea

Usar una distribución auxiliar $q_\phi(z \mid x)$ que proponga z **plausibles para cada** x: el **codificador**. Esto es **inferencia variacional** (como en los modelos gráficos probabilísticos), pero *amortizada* por una red.

La Cota Inferior de la Evidencia (ELBO)

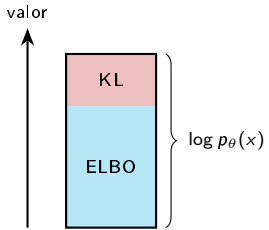
Para cualquier $q_\phi(z \mid x)$, usando la desigualdad de Jensen:

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi(z|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \geq \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \equiv \mathcal{L}(\theta, \phi; x)$$

$$\log p_\theta(x) = \underbrace{\mathcal{L}(\theta, \phi; x)}_{\text{ELBO}} + \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)}_{\geq 0}$$

Lectura

- La brecha es exactamente el KL entre la aproximación y el posterior verdadero.
- Maximizar en $\phi \Rightarrow q_\phi$ se acerca al posterior.
- Maximizar en $\theta \Rightarrow$ sube la verosimilitud.



El ELBO como Reconstrucción + Regularización

Reescribiendo $p_\theta(x, z) = p_\theta(x \mid z)\, p(z)$:

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstrucción}} - \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{regularización}}$$

Término de reconstrucción

- Con $p_\theta(x \mid z) = \mathcal{N}(f_\theta(z), \sigma^2 I)$:

$$\log p_\theta(x \mid z) = -\tfrac{1}{2\sigma^2}\|x - f_\theta(z)\|^2 + \text{cte}$$

- Es el **error cuadrático** del autoencoder.
- Bernoulli $\Rightarrow$ entropía cruzada binaria.

Término KL

- Empuja cada $q_\phi(z \mid x)$ hacia el prior $\mathcal{N}(0, I)$.
- Rellena los "huecos" del espacio latente.
- Penaliza códigos que memorizan cada x.

Compromiso

Reconstruir bien pide codificar mucha información de x en z; el KL pide que z no se aleje del prior. El VAE equilibra **fidelidad** y **regularidad** del espacio latente.

Codificador Gaussiano y KL en Forma Cerrada

Elegimos un codificador gaussiano diagonal:

$$q_\phi(z \mid x) = \mathcal{N}\big(z;\ \mu_\phi(x),\ \mathrm{diag}(\sigma^2_\phi(x))\big)$$

La red g_ϕ devuelve dos vectores: $\mu_\phi(x) \in \mathbb{R}^d$ y $\log \sigma^2_\phi(x) \in \mathbb{R}^d$ (el logaritmo garantiza $\sigma^2 > 0$).

KL entre dos gaussianas (por dimensión)

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)\big) = \frac{1}{2}\Big(\mu^2 + \sigma^2 - \log \sigma^2 - 1\Big)$$

$$\Rightarrow \quad \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) = \frac{1}{2}\sum_{j=1}^{d}\Big(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\Big)$$

El KL se calcula **exactamente**; sólo el término de reconstrucción necesita Monte Carlo (en la práctica, basta $S = 1$ muestra por dato).

El Truco de Reparametrización

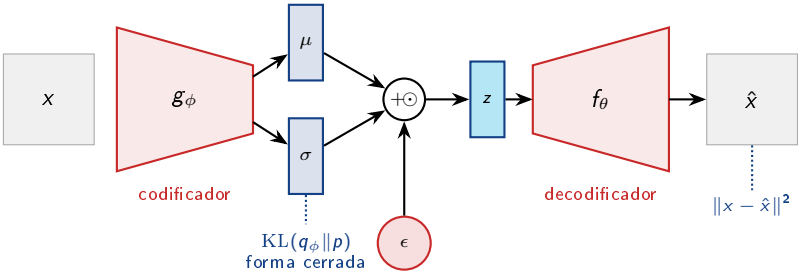
Problema: necesitamos $\nabla_\phi \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$, pero el muestreo $z \sim q_\phi$ **no es diferenciable** respecto a ϕ.



$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I) \quad \Longrightarrow \quad \nabla_\phi \mathbb{E}_{q_\phi}\big[h(z)\big] = \mathbb{E}_\epsilon\Big[\nabla_\phi\, h\big(\mu_\phi(x) + \sigma_\phi(x) \odot \epsilon\big)\Big]$$

La aleatoriedad se traslada a una variable ϵ que **no depende de** ϕ: el gradiente pasa por μ y σ con backpropagation estándar.

Arquitectura Completa del VAE



$$\mathcal{L}_{\text{VAE}}(x) = \frac{1}{2\sigma^2}\left\|x - f_\theta(\mu_\phi(x) + \sigma_\phi(x) \odot \epsilon)\right\|^2 + \frac{1}{2}\sum_{j=1}^{d}\left(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\right)$$

Entrenamiento: minimizar $\mathcal{L}_{\text{VAE}}$ ($= -\text{ELBO}$) con Adam sobre minibatches.

Generación: descartar g_ϕ; $z \sim \mathcal{N}(0, I)$, $\hat{x} = f_\theta(z)$.

Implementación en PyTorch

```
class VAE(nn.Module):
  def __init__(self, D=784, H=400, d=20):
    super().__init__()
    self.enc = nn.Sequential(
        nn.Linear(D, H), nn.ReLU())
    self.mu  = nn.Linear(H, d)
    self.lv  = nn.Linear(H, d)  # log var
    self.dec = nn.Sequential(
        nn.Linear(d, H), nn.ReLU(),
        nn.Linear(H, D), nn.Sigmoid())

  def forward(self, x):
    h = self.enc(x)
    mu, lv = self.mu(h), self.lv(h)
    eps = torch.randn_like(mu)
    z = mu + torch.exp(0.5 * lv) * eps
    return self.dec(z), mu, lv
```

```
def loss_fn(xh, x, mu, lv):
  rec = F.binary_cross_entropy(
          xh, x, reduction='sum')
  kl = -0.5 * torch.sum(
         1 + lv - mu**2 - lv.exp())
  return rec + kl

# generar
z = torch.randn(64, 20)
x_new = model.dec(z)
```

Notas

- MNIST binarizado $\Rightarrow$ Bernoulli $\Rightarrow$ BCE.
- Se predice $\log \sigma^2$ por estabilidad numérica.
- Para imágenes reales: codificador/decodificador convolucionales.

Propiedades y Limitaciones del VAE

Fortalezas

- Entrenamiento **estable** (una sola pérdida, sin minimax).
- Espacio latente **continuo y estructurado**: interpolación $z_t = (1-t)z_a + tz_b$.
- Codificador útil: inferencia de z dado x.
- Cota de la log-verosimilitud $\Rightarrow$ comparar modelos, anomalías.

Debilidades

- Muestras **borrosas**: la verosimilitud gaussiana promedia modos (pérdida L_2).
- La brecha $\mathrm{KL}(q_\phi \| p_\theta(z|x))$ limita la calidad.
- **Colapso del posterior**: con decodificadores muy potentes, $q_\phi(z|x) \approx p(z)$ y z se ignora.

β-VAE y Representaciones Desenredadas

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi}\big[\log p_\theta(x \mid z)\big] - \beta\,\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

$\beta > 1$: latentes más independientes e interpretables (cada z_j controla un factor: pose, color, tamaño), a costa de reconstrucción. $\beta < 1$: mejor reconstrucción.

Variantes Modernas del VAE

VAE Condicional

- $q_\phi(z \mid x, y)$, $p_\theta(x \mid z, y)$.
- Generar dada una clase o atributo y.

VQ-VAE

- Latente **discreto**: z se cuantiza al vector más cercano de un diccionario.
- Evita el colapso del posterior.
- Tokeniza imágenes para transformers.

VAE Jerárquico

- Varias capas de latentes $z_1, \ldots, z_L$ (NVAE).
- La difusión puede verse como un VAE jerárquico con codificador **fijo**.

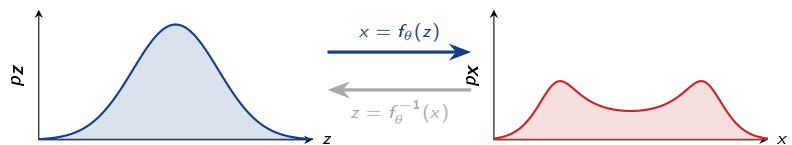
El VAE en los Sistemas Actuales

Stable Diffusion es un **modelo de difusión latente**: un autoencoder variacional comprime la imagen $512 \times 512 \times 3$ a un latente $64 \times 64 \times 4$, y la difusión opera **en ese espacio latente**. El VAE sigue siendo una pieza central de la IA generativa.

Flujos Normalizantes: Idea

Idea: transformar una distribución simple p_Z en una compleja mediante una función **invertible y diferenciable** f_θ.

$$z \sim p_Z(z) = \mathcal{N}(0, I), \qquad x = f_\theta(z), \qquad z = f_\theta^{-1}(x)$$



Ejemplo: $f(z) = z + 1.5\tanh(2z)$ estira el centro y convierte una gaussiana en una densidad **bimodal**. La masa de probabilidad se conserva: donde f estira, la densidad baja.

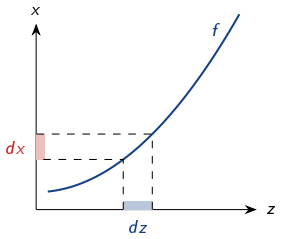
Fórmula de Cambio de Variables

Caso 1D

La probabilidad de un intervalo se conserva:

$$p_X(x)\,|dx| = p_Z(z)\,|dz|$$

$$\Rightarrow\; p_X(x) = p_Z\big(f^{-1}(x)\big)\left|\frac{df^{-1}(x)}{dx}\right|$$



Caso multivariado ($x, z \in \mathbb{R}^D$)

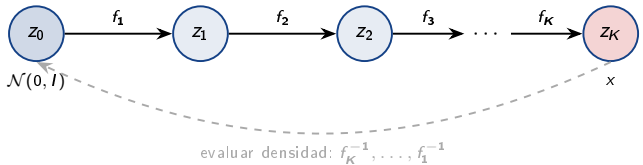
El factor de escala es el valor absoluto del **determinante del jacobiano**:

$$\log p_X(x) = \log p_Z\big(f_\theta^{-1}(x)\big) + \log\left|\det \frac{\partial f_\theta^{-1}(x)}{\partial x}\right| = \log p_Z(z) - \log\left|\det \frac{\partial f_\theta(z)}{\partial z}\right|$$

¡Verosimilitud **exacta**! Pero z debe tener la **misma dimensión** que x (no hay cuello de botella).

Componer Transformaciones

Una sola f simple no basta. Componemos K capas invertibles:



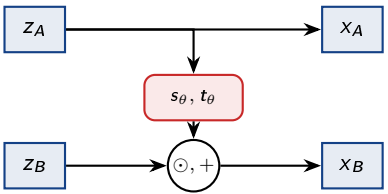
$$x = f_K \circ \cdots \circ f_1(z_0) \qquad \Rightarrow \qquad \log p_X(x) = \log p_Z(z_0) - \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

Requisitos de cada capa f_k

1. **Invertible** (y de preferencia con inversa barata).
2. Determinante del jacobiano **barato**: el costo general de det es $O(D^3)$, inviable para $D \approx 10^5$.
3. **Expresiva**: la composición debe poder representar distribuciones complejas.

Capas de Acoplamiento Afín (RealNVP)

Dividir $z = (z_A, z_B)$. Una mitad pasa intacta y **parametriza** la transformación de la otra:



$$x_A = z_A$$
$$x_B = z_B \odot \exp\big(s_\theta(z_A)\big) + t_\theta(z_A)$$

s_θ, t_θ: redes **arbitrarias** (CNN, ResNet). Nunca se invierten.

Inversa trivial

$$z_A = x_A, \quad z_B = \big(x_B - t_\theta(x_A)\big) \odot \exp\big(-s_\theta(x_A)\big)$$

Jacobiano triangular

$$\frac{\partial x}{\partial z} = \begin{pmatrix} I & 0 \\ * & \operatorname{diag}(e^{s_\theta(z_A)}) \end{pmatrix}$$
$$\Rightarrow \log|\det| = \textstyle\sum_j s_\theta(z_A)_j$$

Costo $O(D)$. Se **alterna** qué mitad se transforma (o se permutan canales) para que todas las variables se mezclen.

Flujo autorregresivo enmascarado (MAF) e inverso (IAF)

Cada dimensión se transforma condicionada en las **anteriores**:

$$x_i = z_i \exp\left(s_i(x_{1:i-1})\right) + t_i(x_{1:i-1}) \qquad \Rightarrow \qquad \text{jacobiano triangular, } \log|\det| = \sum_i s_i$$

- **MAF**: densidad rápida (paralela, con máscaras tipo MADE), muestreo lento (D pasos secuenciales).
- **IAF**: lo contrario. Muestreo rápido, densidad lenta. Útil como q_ϕ en un VAE.

Glow (2018)

- ActNorm + convolución invertible 1×1 (permutación aprendida) + acoplamiento afín.
- Arquitectura multiescala.
- $\log|\det W|$ con descomposición LU.

Flujos continuos

- $\frac{dz(t)}{dt} = v_\theta(z(t), t)$: una ODE neuronal.
- $\frac{d \log p(z(t))}{dt} = -\mathrm{tr}\left(\frac{\partial v_\theta}{\partial z}\right)$.
- Base del **flow matching** moderno.

Entrenamiento y Balance de los Flujos

$$\theta^* = \arg\max_{\theta} \sum_{i=1}^{N} \left[\log p_Z\left(f_\theta^{-1}(x^{(i)})\right) + \log \left| \det \frac{\partial f_\theta^{-1}}{\partial x}\left(x^{(i)}\right) \right| \right]$$

Máxima verosimilitud **exacta** por descenso de gradiente. Se reporta en *bits por dimensión*: $-\log_2 p(x)/D$.

Ventajas

- Log-verosimilitud **exacta**.
- Inferencia exacta $z = f^{-1}(x)$.
- Muestreo en una pasada (acoplamiento).
- Entrenamiento estable.

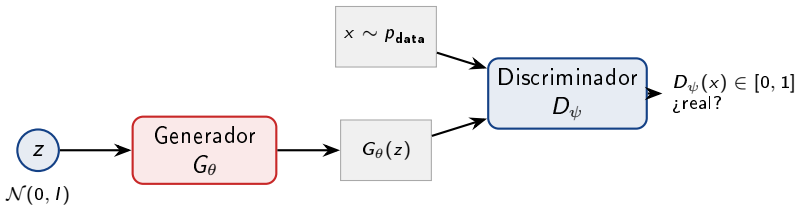
Desventajas

- $\dim z = \dim x$: sin compresión.
- Restricciones arquitectónicas $\Rightarrow$ muchos parámetros para igualar a otros modelos.
- Calidad visual inferior a GAN y difusión.
- Topología: un homeomorfismo no puede "romper" una gaussiana en componentes separados.

Usos actuales: estimación de densidad, detección de anomalías, distribuciones variacionales flexibles, muestreo en física (rejillas, moléculas) e inferencia bayesiana amortizada.

GAN: Aprender sin Evaluar la Densidad

Idea (Goodfellow et al., 2014): no modelar $p_\theta(x)$ explícitamente; entrenar un generador G contra un **discriminador** D que intenta distinguir datos reales de falsos.



$$\min_\theta \max_\psi \; V(G,D) = \mathbb{E}_{x\sim p_{\text{data}}}\big[\log D_\psi(x)\big] + \mathbb{E}_{z\sim p(z)}\big[\log\big(1 - D_\psi(G_\theta(z))\big)\big]$$

D maximiza: clasificador binario con entropía cruzada.

G minimiza: engañar a D. Sólo necesita **muestrear**, nunca evaluar $p_\theta(x)$.

¿Qué Divergencia Minimiza una GAN?

Discriminador óptimo para G fijo: $D^*(x) = \dfrac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$. Sustituyendo en V:

$$V(G, D^*) = -\log 4 + 2\,\mathrm{JS}\big(p_{\text{data}} \,\|\, p_G\big) \qquad \Rightarrow \qquad \text{óptimo global en } p_G = p_{\text{data}},\ D^* = \tfrac{1}{2}$$

$\mathrm{JS}(p\|q) = \frac{1}{2}\mathrm{KL}\big(p \,\|\, \frac{p+q}{2}\big) + \frac{1}{2}\mathrm{KL}\big(q \,\|\, \frac{p+q}{2}\big)$: divergencia de Jensen–Shannon, simétrica y acotada.

Pérdida no saturante (práctica)

Al inicio $D(G(z)) \approx 0$ y $\log(1 - D(G(z)))$ tiene gradiente casi nulo. En la práctica G maximiza

$$\mathbb{E}_z\big[\log D_\psi(G_\theta(z))\big]$$

Algoritmo (alternado)

1. k pasos de ascenso en ψ con minibatch real + falso.
2. 1 paso de descenso en θ.
3. Repetir.

Dificultades y Mejoras de las GAN

Problemas

- **Colapso de modos**: G produce pocas variantes que engañan a D.
- **Inestabilidad**: juego minimax sin garantía de convergencia; oscilaciones.
- **Gradientes que desaparecen**: si los soportes no se solapan, JS es constante ($\log 2$).
- Sin verosimilitud: difícil de evaluar (se usa FID).

Soluciones

- **WGAN**: distancia de Wasserstein (Earth Mover) con crítico 1-Lipschitz:

$$W(p_{\text{data}}, p_G) = \sup_{\|D\|_L \leq 1} \mathbb{E}_{p_{\text{data}}}[D(x)] - \mathbb{E}_{p_G}[D(x)]$$

- Penalización de gradiente (WGAN-GP), normalización espectral.
- **DCGAN**: guías arquitectónicas convolucionales.
- **StyleGAN**: latente mapeado a estilos por capa; fotorrealismo.

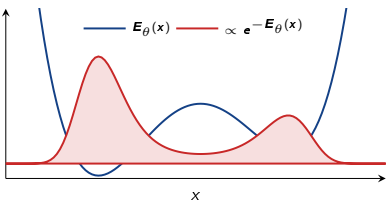
Conexión con la Clase 2.5

CycleGAN para **adaptación de dominio**: dos generadores $G : A \to B$, $F : B \to A$ con consistencia cíclica $\|F(G(x)) - x\|$.

Modelos Basados en Energía (EBM)

Idea: renunciar a la normalización. Basta una red escalar $E_\theta : \mathbb{R}^D \to \mathbb{R}$ (la **energía**): baja para datos plausibles, alta para el resto.

$$p_\theta(x) = \frac{\exp\big(-E_\theta(x)\big)}{Z_\theta}, \qquad Z_\theta = \int \exp\big(-E_\theta(x)\big)\, dx \quad \text{(función de partición)}$$



Ventajas

- **Máxima flexibilidad**: cualquier arquitectura que produzca un escalar.
- Composición: $E = E_1 + E_2$ (producto de expertos).
- Física estadística: distribución de Gibbs/Boltzmann.

El precio

Z_θ es una integral en $\mathbb{R}^D$: **intratable**. No podemos evaluar $p_\theta(x)$ ni muestrear directamente.

Gradiente de la Log-Verosimilitud

Aunque Z_θ es intratable, su gradiente tiene una forma elegante:

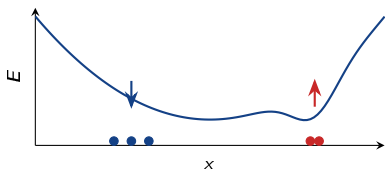
$$\nabla_\theta \log Z_\theta = \frac{1}{Z_\theta} \int -\nabla_\theta E_\theta(x)\, e^{-E_\theta(x)}\, dx = -\mathbb{E}_{x \sim p_\theta}\left[\nabla_\theta E_\theta(x)\right]$$

$$\nabla_\theta \mathbb{E}_{p_{\text{data}}}\left[\log p_\theta(x)\right] = \underbrace{-\mathbb{E}_{x \sim p_{\text{data}}}\left[\nabla_\theta E_\theta(x)\right]}_{\text{fase positiva}} + \underbrace{\mathbb{E}_{x' \sim p_\theta}\left[\nabla_\theta E_\theta(x')\right]}_{\text{fase negativa}}$$

Interpretación

- **Bajar** la energía en los datos reales.
- **Subir** la energía en las muestras del modelo ("alucinaciones").
- Equilibrio: $p_\theta = p_{\text{data}}$.

El problema se traslada a **muestrear** $x' \sim p_\theta$.



datos: bajar E. muestras del modelo: subir E.

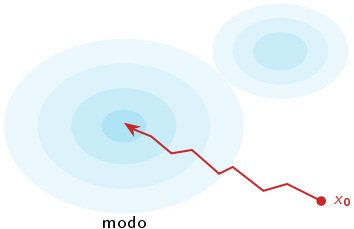
Muestreo con Dinámica de Langevin

Observación clave: el **score** $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$ **no depende de** Z_θ.

$$x_{k+1} = x_k - \frac{\eta}{2}\nabla_x E_\theta(x_k) + \sqrt{\eta}\,\epsilon_k, \qquad \epsilon_k \sim \mathcal{N}(0, I)$$

MCMC por gradiente

- Descenso de gradiente sobre la energía + **ruido**.
- Con $\eta \to 0$ y $K \to \infty$, $x_K \sim p_\theta$.
- El ruido evita quedar atrapado en un mínimo: explora la distribución completa.
- $\nabla_x E_\theta$ se obtiene con autograd.



Costo

Cada paso de entrenamiento requiere decenas o cientos de pasos de Langevin: **lento**, y mezcla mal entre modos separados.

Entrenar un EBM: Divergencia Contrastiva

Algoritmo (Contrastive Divergence / PCD)

1. Tomar un minibatch real $\{x^{(i)}\}$.
2. Generar negativos $\{x'^{(i)}\}$ con K pasos de Langevin, partiendo de:
   - los datos (CD-K, Hinton 2002), o
   - un **buffer de reinicio** con cadenas persistentes (PCD), o ruido uniforme.
3. Actualizar θ minimizando la pérdida contrastiva:

$$\mathcal{L}_{\mathrm{CD}}(\theta) = \frac{1}{B}\sum_{i=1}^{B} E_\theta\big(x^{(i)}\big) - \frac{1}{B}\sum_{i=1}^{B} E_\theta\big(\mathrm{sg}[x'^{(i)}]\big) \;+\; \alpha \cdot \mathrm{reg}$$

$\mathrm{sg}[\cdot]$: *stop-gradient* (las muestras son constantes). El término de regularización (p.ej. $\alpha(E(x)^2 + E(x')^2)$) evita que las energías diverjan.

Historia: máquinas de Boltzmann restringidas (RBM, 2006). **Moderno**: EBMs con CNN (Du & Mordatch, 2019) para generación, inpainting y composición de conceptos.

Alternativas sin Muestreo: Score Matching y NCE

Score Matching (Hyvärinen, 2005)

Igualar los **scores** en vez de las densidades:

$$\tfrac{1}{2}\mathbb{E}_{p_{\text{data}}}\left\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\right\|^2,$$

con $s_\theta = -\nabla_x E_\theta$. Integrando por partes:

$$\mathbb{E}_{p_{\text{data}}}\left[\tfrac{1}{2}\|s_\theta(x)\|^2 + \operatorname{tr}\left(\nabla_x s_\theta(x)\right)\right]$$

No requiere Z_θ ni muestras. La traza es cara en alta dimensión.

Noise-Contrastive Estimation

- Clasificar datos reales vs. ruido $q(x)$ conocido.
- Logit: $\log p_\theta(x) - \log q(x)$, con $\log Z$ como **parámetro aprendido**.
- Regresión logística $\Rightarrow$ estimador consistente.
- Precursor de las pérdidas contrastivas (InfoNCE).

Denoising Score Matching (Vincent, 2011)

Perturbar $\tilde{x} = x + \sigma\epsilon$ y regresar el ruido: $\mathbb{E}\left\|s_\theta(\tilde{x}) + \frac{\tilde{x}-x}{\sigma^2}\right\|^2$. Con muchos niveles de ruido $\sigma_1 > \cdots > \sigma_L$ y Langevin para muestrear $\Rightarrow$ **modelos de difusión**.

Tu Clasificador es Secretamente un EBM

Un clasificador con logits $f_\theta(x) \in \mathbb{R}^K$ define $p_\theta(y \mid x) = \mathrm{softmax}(f_\theta(x))_y$. Reinterpretando los logits como energías negativas (JEM, Grathwohl et al., 2020):

$$p_\theta(x, y) = \frac{\exp\left(f_\theta(x)[y]\right)}{Z_\theta} \quad \Rightarrow \quad p_\theta(x) = \frac{\sum_y \exp\left(f_\theta(x)[y]\right)}{Z_\theta}, \quad E_\theta(x) = -\log \sum_y \exp\left(f_\theta(x)[y]\right)$$

Consecuencias

- $p(y \mid x)$ **no cambia**: Z_θ se cancela.
- Pero la misma red define un modelo generativo de x.
- Entrenar $\log p(y|x) + \log p(x)$ mejora calibración y detección de datos fuera de distribución.

Conexión

- El softmax es una distribución de Gibbs sobre clases.
- $\mathrm{logsumexp}$ de los logits $\approx$ "confianza" del modelo: base de detectores OOD por energía.
- Une las clases discriminativas de la unidad con los modelos generativos.

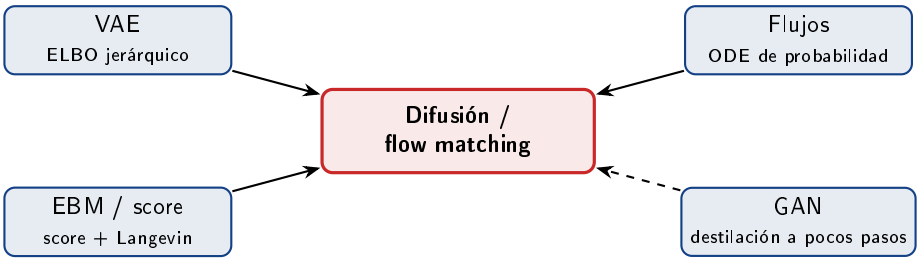
Comparación de Familias

	VAE	Flujos	GAN	EBM
Densidad $p_\theta(x)$	cota (ELBO)	exacta	ninguna	sin normalizar
Muestreo	rápido ✓	rápido ✓	rápido ✓	lento (MCMC) ×
Calidad de muestras	borrosa –	media –	alta ✓	media/alta –
Cobertura de modos	buena ✓	buena ✓	colapso ×	buena ✓
Latente útil	compacto ✓	dim $= D$ –	sin codificador –	no ×
Estabilidad	alta ✓	alta ✓	baja ×	media –
Restricción arquitectónica	baja	**alta**	baja	**ninguna**
Estrategia ante Z_θ	cota variacional	jacobiano	evitar la densidad	score / MCMC

El "trilema" generativo

Ninguna familia clásica logra a la vez **alta calidad**, **cobertura de modos** y **muestreo rápido**. Los modelos de difusión y flow matching sacrifican velocidad para obtener las dos primeras.

Hacia la Difusión: Lo Que Hereda de Cada Familia



- **VAE**: la difusión es un VAE jerárquico con T latentes y codificador fijo (agregar ruido).
- **EBM**: el denoising score matching entrena $s_\theta \approx \nabla_x \log p_\sigma(x)$; se muestrea con Langevin.
- **Flujos**: la ODE de flujo de probabilidad da verosimilitud exacta; flow matching entrena un flujo continuo por regresión.
- **GAN**: discriminadores adversarios para destilar la difusión a pocos pasos.

Conclusiones

- Un modelo generativo aproxima p_{data}; la máxima verosimilitud equivale a minimizar $\mathrm{KL}(p_{\text{data}} \| p_\theta)$. El obstáculo común es la **normalización**.
- **VAE**: modelo de variable latente entrenado con el ELBO (reconstrucción − KL) y el truco de reparametrización. Latente estructurado; muestras algo borrosas.
- **Flujos normalizantes**: transformaciones invertibles con jacobiano barato (acoplamiento, autorregresivos). Verosimilitud exacta, a costa de restricciones arquitectónicas.
- **GAN**: juego minimax generador–discriminador. Alta calidad visual; inestable y con colapso de modos.
- **EBM**: densidad no normalizada $e^{-E_\theta(x)}$, máxima flexibilidad. Se entrena con fases positiva/negativa (Langevin) o score matching, que conduce a la **difusión**.

Para el laboratorio

Implementar un VAE convolucional en MNIST/Fashion-MNIST: visualizar el espacio latente 2D, interpolar entre dígitos y comparar con muestras de un flujo RealNVP en un conjunto 2D (*two moons*).

Referencias

- S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Caps. 14 (no supervisado), 15 (GAN), 16 (flujos normalizantes), 17 (VAE), 18 (difusión). `udlbook.github.io`
- A. Torralba, P. Isola & W. Freeman, *Foundations of Computer Vision*, MIT Press, 2024. Caps. sobre modelos generativos.
- D. P. Kingma & M. Welling, "Auto-Encoding Variational Bayes", ICLR 2014.
- D. J. Rezende & S. Mohamed, "Variational Inference with Normalizing Flows", ICML 2015.
- L. Dinh, J. Sohl-Dickstein & S. Bengio, "Density Estimation using Real NVP", ICLR 2017.
- D. P. Kingma & P. Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions", NeurIPS 2018.
- I. Goodfellow et al., "Generative Adversarial Nets", NeurIPS 2014. M. Arjovsky et al., "Wasserstein GAN", ICML 2017.
- Y. LeCun et al., "A Tutorial on Energy-Based Learning", 2006.
- Y. Du & I. Mordatch, "Implicit Generation and Modeling with Energy-Based Models", NeurIPS 2019.
- W. Grathwohl et al., "Your Classifier is Secretly an Energy Based Model", ICLR 2020.
- Y. Song & D. P. Kingma, "How to Train Your Energy-Based Models", arXiv:2101.03288, 2021.